Random Binary Search Trees
Treaps

COMP 2402

Abstract Data Types & Algorithms

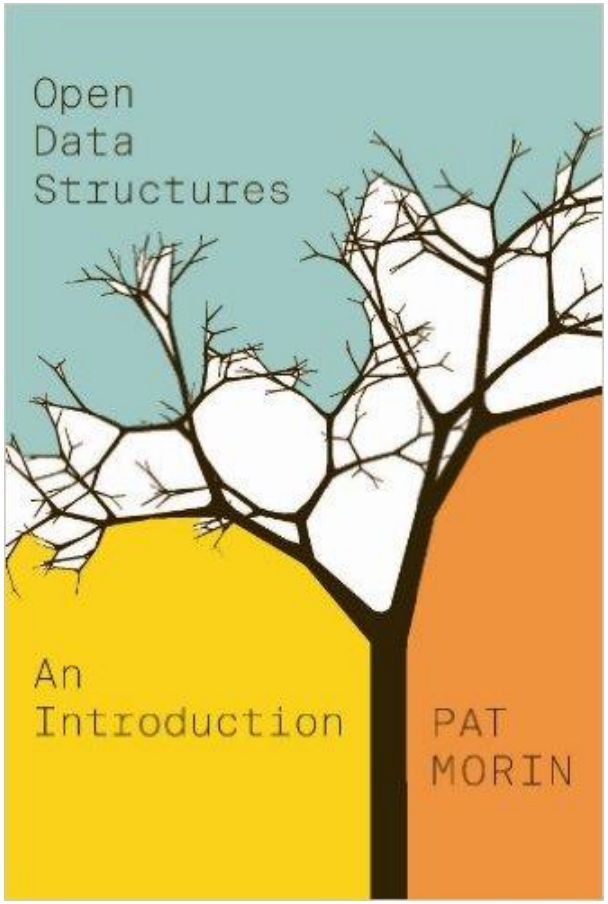
Readings

Today's class

- Binary Search Trees
  - Chapter 6, 7.1

Next Class

- Treaps
  - Chapter 7.2



Binary Search Trees

In a **binary search tree**, each node stores some data from some **total order**

The **binary search tree property**: (with distinct items)

For any node u

1. All data values stored in the subtree u.left are less than u.x
2. All data values stored in the subtree ru.right are greater then u.x

Binary Search Trees

Theorem 6.1: **BinarySearchTree** implements the SSet interface and supports the operations add(x), remove(x), and find(x) in $O(n)$ time per operation.

How does this compare to the skiplist?

Random Binary Search Trees

What is the worst case we can have for a BST?

What is the best case we can have for a BST?

Random Binary Search Trees

Consider a BST with 10 elements {0,1,…,9}

Random Binary Search Trees

To create a random binary search tree (BST) we take a permutation of $0,1, \ldots n-1$ and add the elements one by one.

We hope that the resulting tree is good.

(We expect the resulting tree to be good.)

Random Binary Search Trees

To create a random binary search tree (BST) we take a permutation of $0,1, \ldots n-1$ and add the elements one by one.

Draw the BST corresponding to the random permutation of 0, …, 10

$$< 4, 2, 8, 6, 7, 1, 10, 9, 0, 3, 5 >$$

Random Binary Search Trees

We want binary search trees that store any data that has a total order.

Why do we just consider the numbers 0,1,…,n-1?

Harmonic Numbers

The k-th harmonic number H_k is the number

$$H_k = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{k} = \sum_{i=1}^{k} \frac{1}{i}$$

$$\log_e(k) < H_k \leq \log_e(k) + 1$$

$$\ln(k) < H_k \leq \ln(k) + 1$$

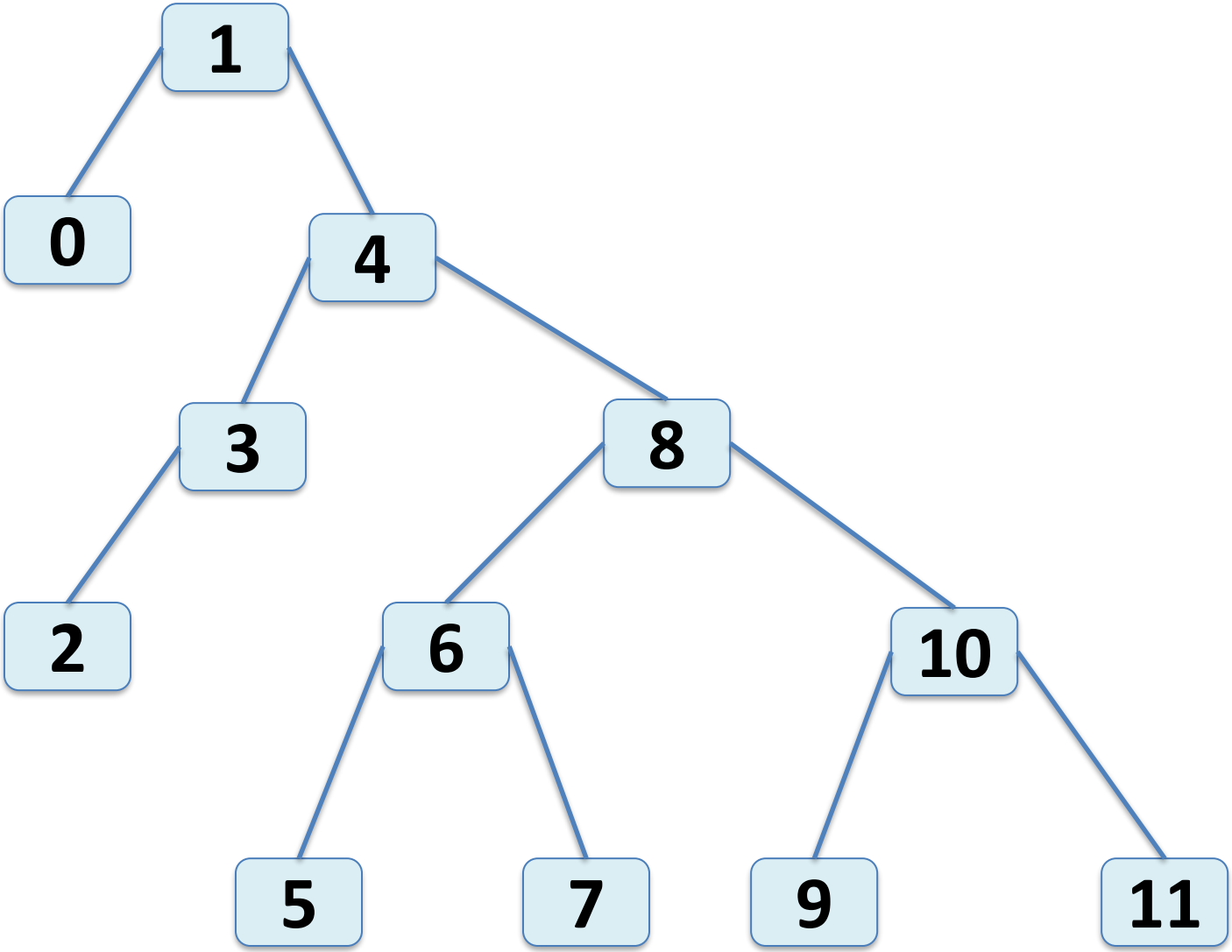
Random Binary Search Trees

Lemma 7.1: In a random binary search tree (BST) of size n, the following holds

1. For any $x \in \{0,1, \dots n-1\}$, the expected length of a search path for x is $H_{x+1} + H_{n-x} - O(1)$

2. For any $x \in (-1, n) \backslash \{0,1, \dots n-1\}$, the expected length of a search path for x is $H_{\lceil x \rceil} + H_{n-\lceil x \rceil}$

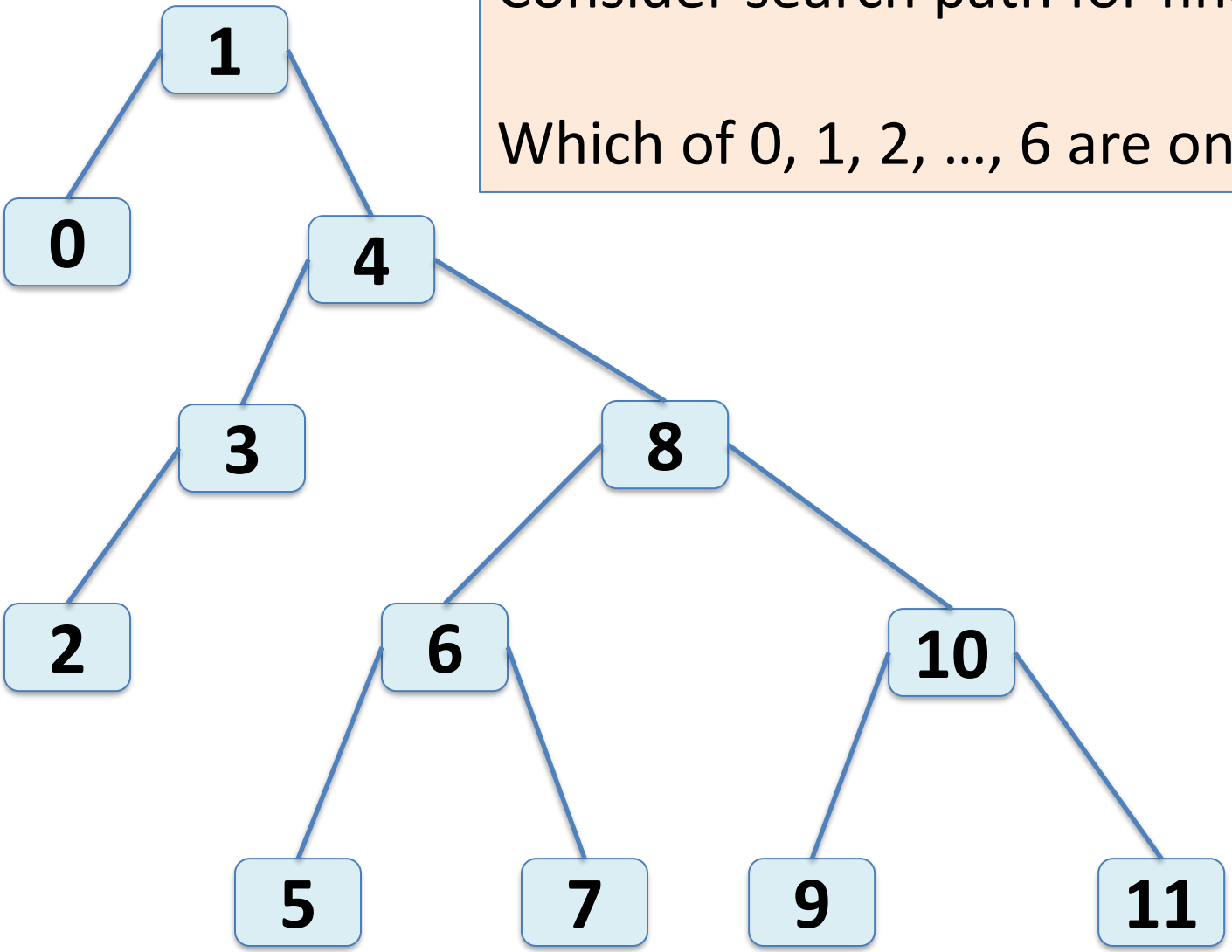
Random Binary Search Trees



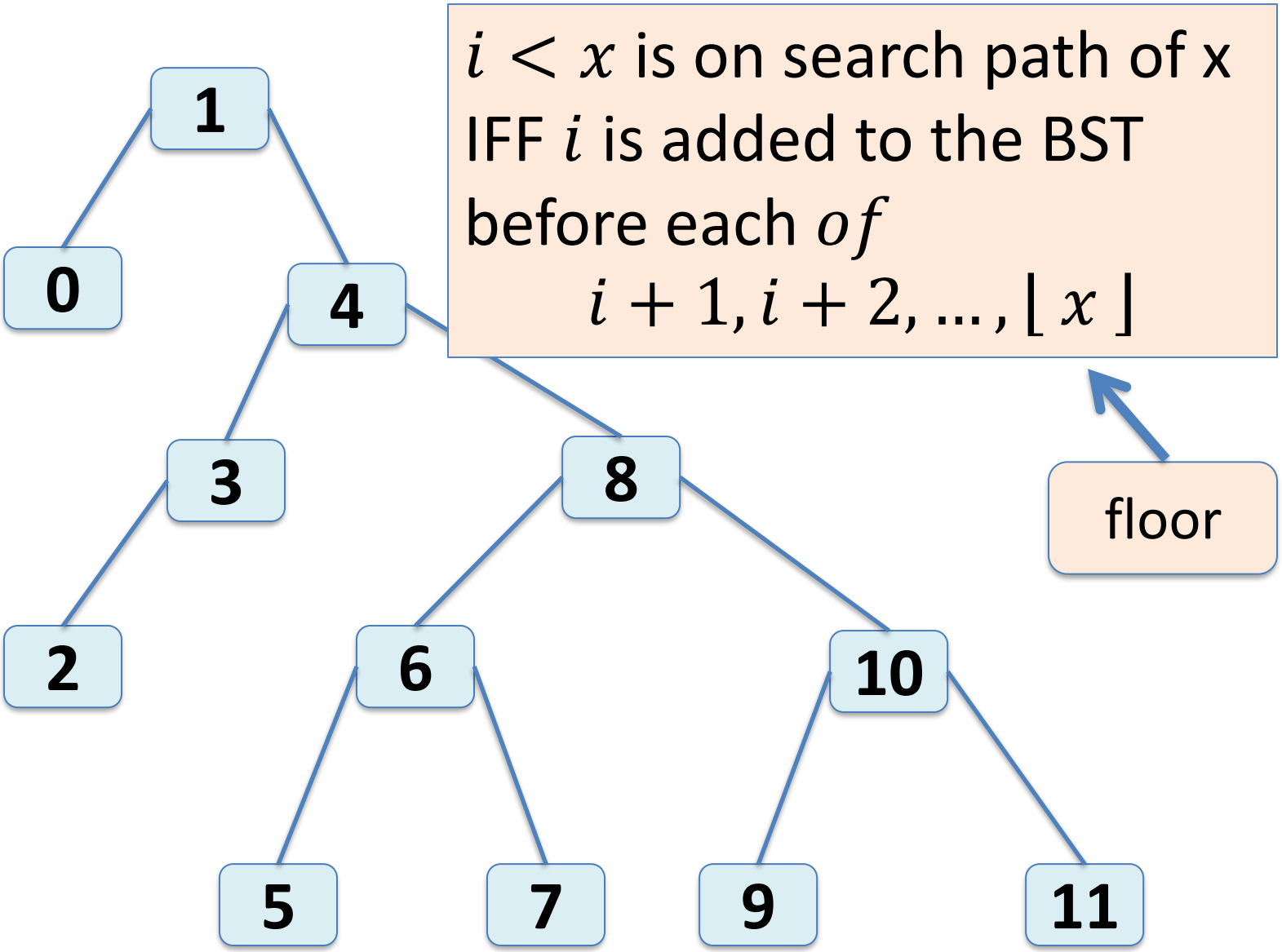
Random Binary Search Trees

Consider search path for find(7)

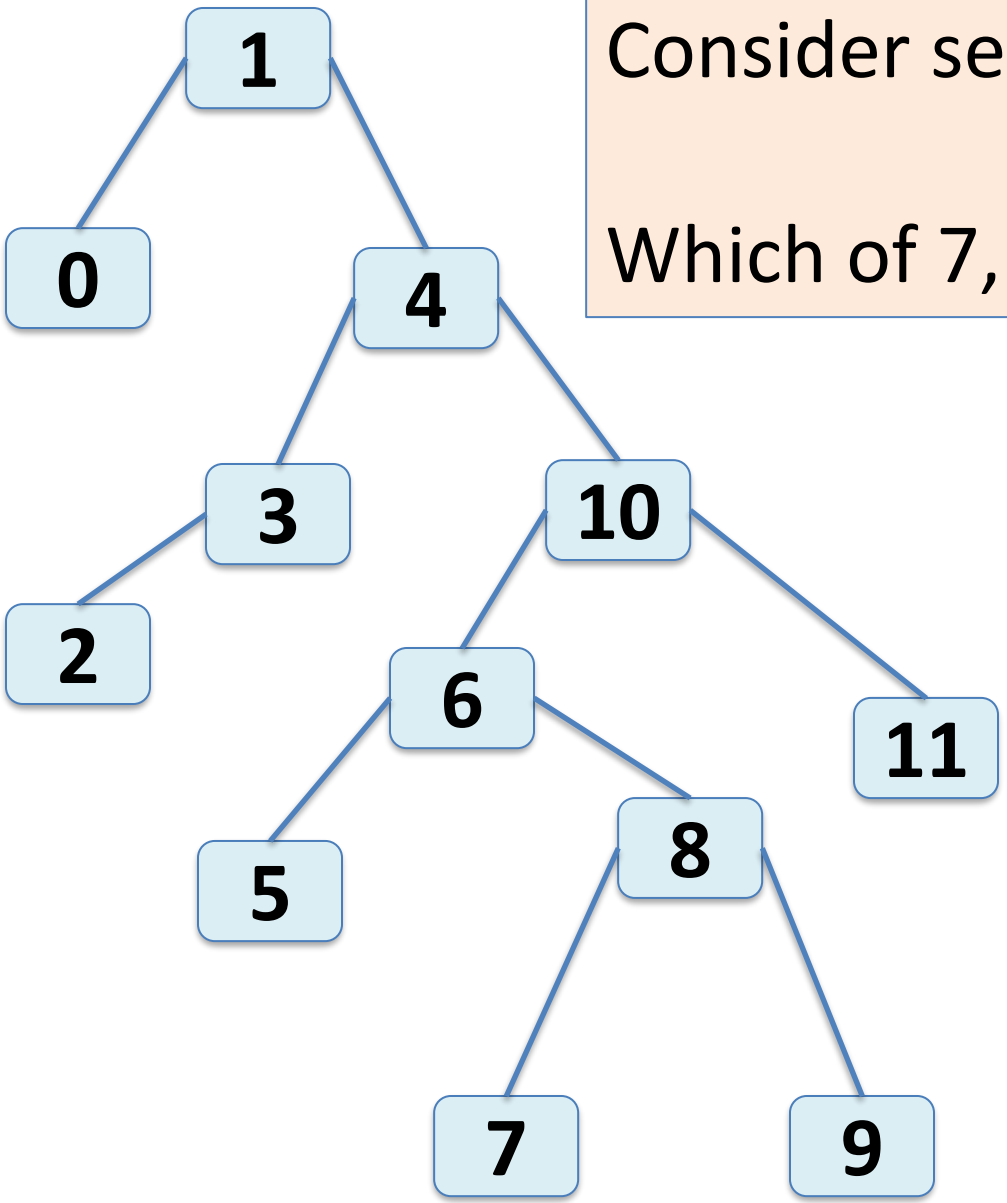
Which of 0, 1, 2, …, 6 are on it?



Random Binary Search Trees



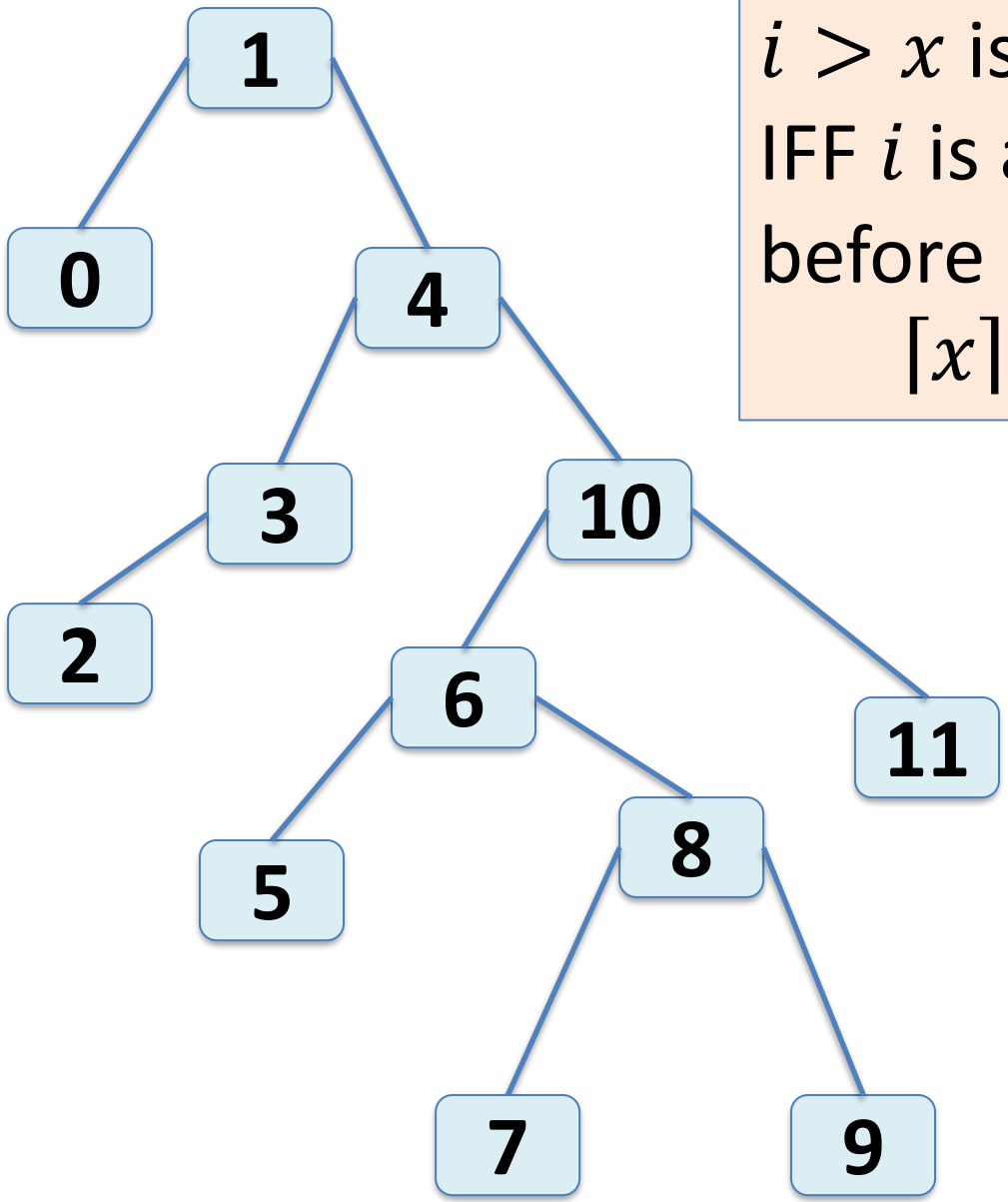
Random Binary Search Trees



Consider search path for find(6)

Which of 7, 8, …, 11 are on it?

Random Binary Search Trees



$i > x$ is on search path of x IFF i is added to the BST before each of

$$\lceil x \rceil, \lceil x \rceil + 1, \ldots i - 1$$

ceiling

Random Binary Search Trees

For $i \in \mathbb{Z}$ to be in the search path of $x \in \mathbb{R}$

- When $i < x$ then i must be added before each of

$$i + 1, i + 2, \ldots, \lfloor x \rfloor$$

- When $i > x$ then i must be added before each of

$$\lceil x \rceil, \lceil x \rceil + 1, \ldots, i - 1$$

Random Binary Search Trees

For $i \in \mathbb{Z}$ to be in the search path of $x \in \mathbb{R}$

- When $i < x$ then i must appear before any of
$$i + 1, i + 2, \dots, \lfloor x \rfloor$$
in the random permutation that generated the BST

- When $i > x$ then i must appear before any of
$$\lceil x \rceil, \lceil x \rceil + 1, \dots, i - 1$$
- in the random permutation that generated the BST

Random Binary Search Trees

What is the probability that i appears before the first or after the last

$$i + 1, i + 2, \dots, \lfloor x \rfloor$$

$$\lceil x \rceil, \lceil x \rceil + 1, \dots, i - 1$$

in the permutation that generated the BST?

Random Binary Search Trees

Look at these subsequences in random permutation

$$\boldsymbol{i}, i+1, i+2, \ldots, \lfloor x \rfloor$$

$$\lceil x \rceil, \lceil x \rceil + 1, \ldots, i-1, \boldsymbol{i}$$

$$\Pr(i \; on \; path \; for \; x) = \begin{cases} \dfrac{1}{\lfloor x \rfloor - i + 1} & when \; i < x \\ \dfrac{1}{i - \lceil x \rceil + 1} & when \; i > x \end{cases}$$

Harmonic Numbers

The k-th harmonic number H_k is the number

$$H_k = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{k} = \sum_{i=1}^{k} \frac{1}{i}$$

$$\log_e(k) < H_k \leq \log_e(k) + 1$$

$$\ln(k) < H_k \leq \ln(k) + 1$$

Random Binary Search Trees

Lemma 7.1: In a random binary search tree (BST) of size n, the following holds

1. For any $x \in \{0,1, \ldots n-1\}$, the expected length of a search path for x is $H_{x+1} + H_{n-x} - O(1)$

2. For any $x \in (-1, n) \backslash \{0,1, \ldots n-1\}$, the expected length of a search path for x is $H_{\lceil x \rceil} + H_{n-\lceil x \rceil}$

Random Binary Search Trees

Lemma 7.1: In a random binary search tree (BST) of size n, the following holds

1. For any $x \in \{0,1$ of a se $n-x - O(1)$

2. $1, n)\backslash\{0,1, \ldots n-1\}$, the ected length of a search path for x is $H_{\lceil x \rceil} + H_{n-\lceil x \rceil}$

Expected search path length is at most 2ln(n) + O(1)

Random Binary Search Trees

What is expected runtime of searching in a random BST?

What the expected runtime of adding an element to a random BST?

Random Binary Search Trees

Theorem 7.1: A random binary search tree can be constructed in time $O(n \ln(n))$ time. In a random binary search tree, the find(x) operation takes $O(\ln(n))$ expected time.

Note: The expected runtimes of the random BST are based on the random permutation used to generate the BST. It does not include adding and removing elements after this tree is created.

TREAP

A **treap** is a binary tree data structure that obeys both the **binary search tree property** and the **heap property**.

Each node has two pieces of information

- x is the data
- p is the priority (which is assigned randomly)

TREAP

Each node has two pieces of information

- x is the data

Binary search tree property: For node u, all data in the subtree u.left is less than u.x and all data in the substree u.right is greater than u.x

TREAP

Each node has two pieces of information

- p is the priority (which is assigned randomly)

Heap property: at every node u, except the root

$$u.parent.p < u.p$$

TREAP

How do we add a node to a treap?

TREAP

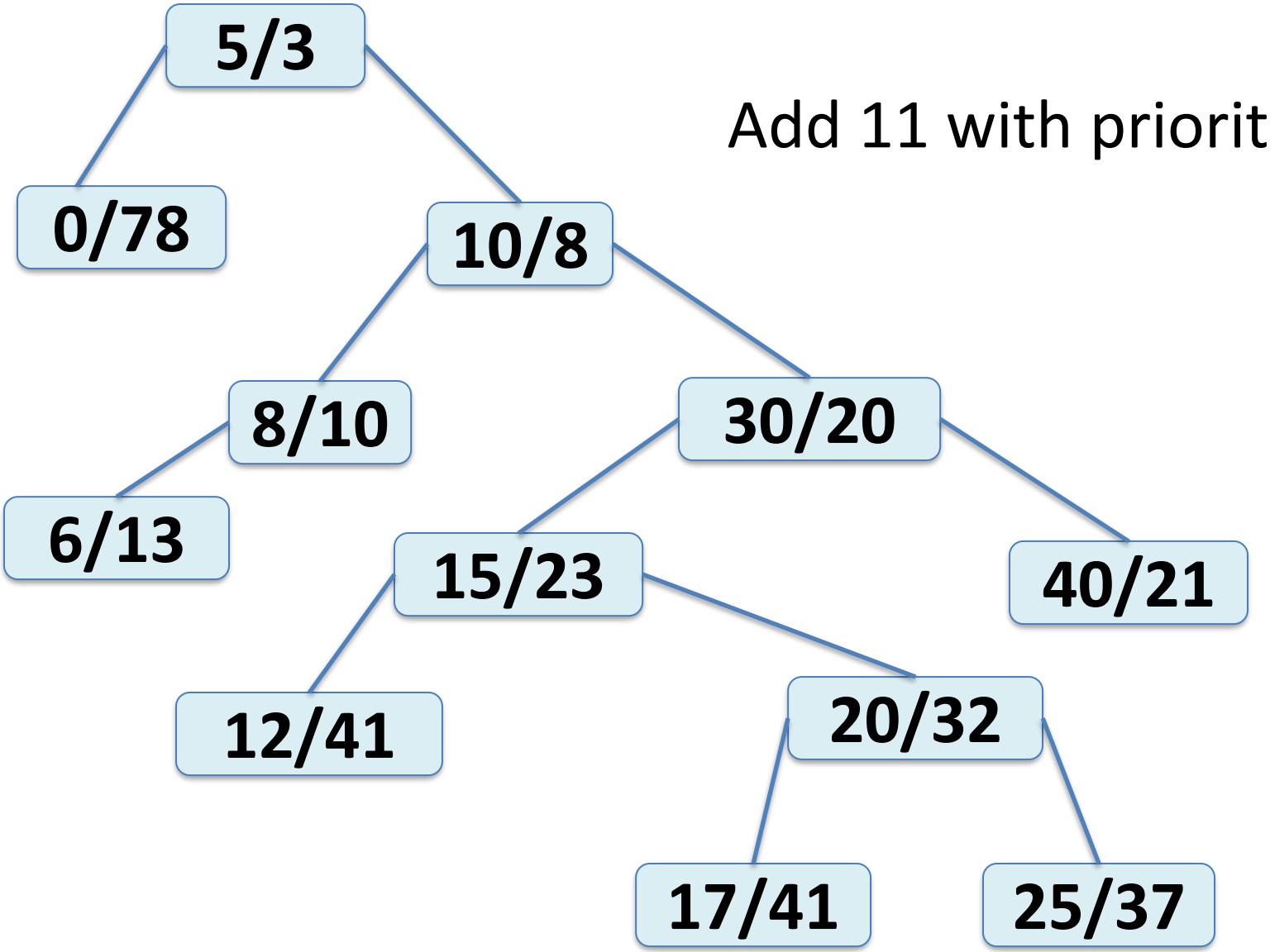
How do we add a node to a treap?

Observation:

Adding a new node, with a random priority, in the way we have seen for binary trees will

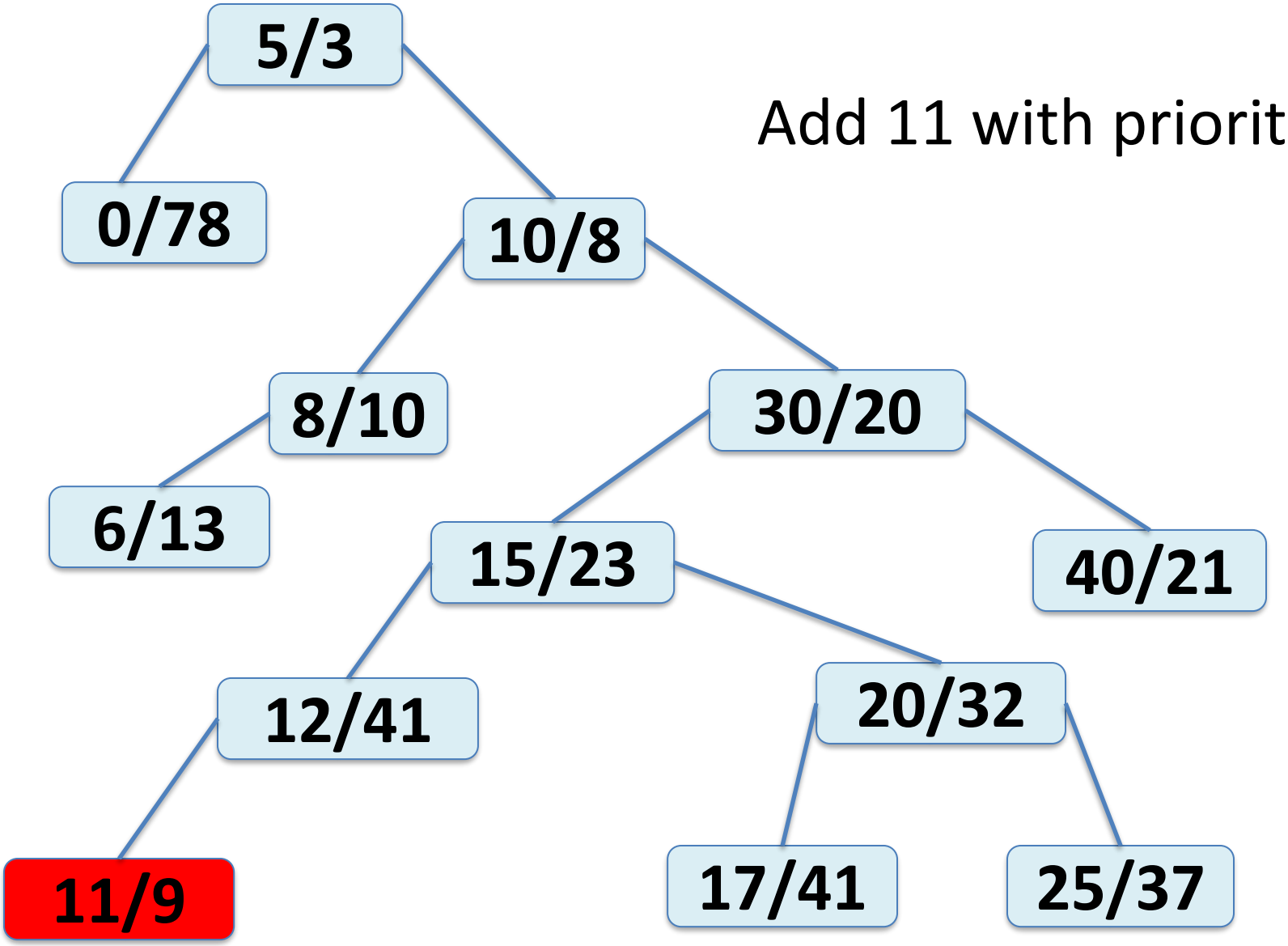
1. always maintain the binary search tree property,
2. likely not maintain the heap property.

TREAP



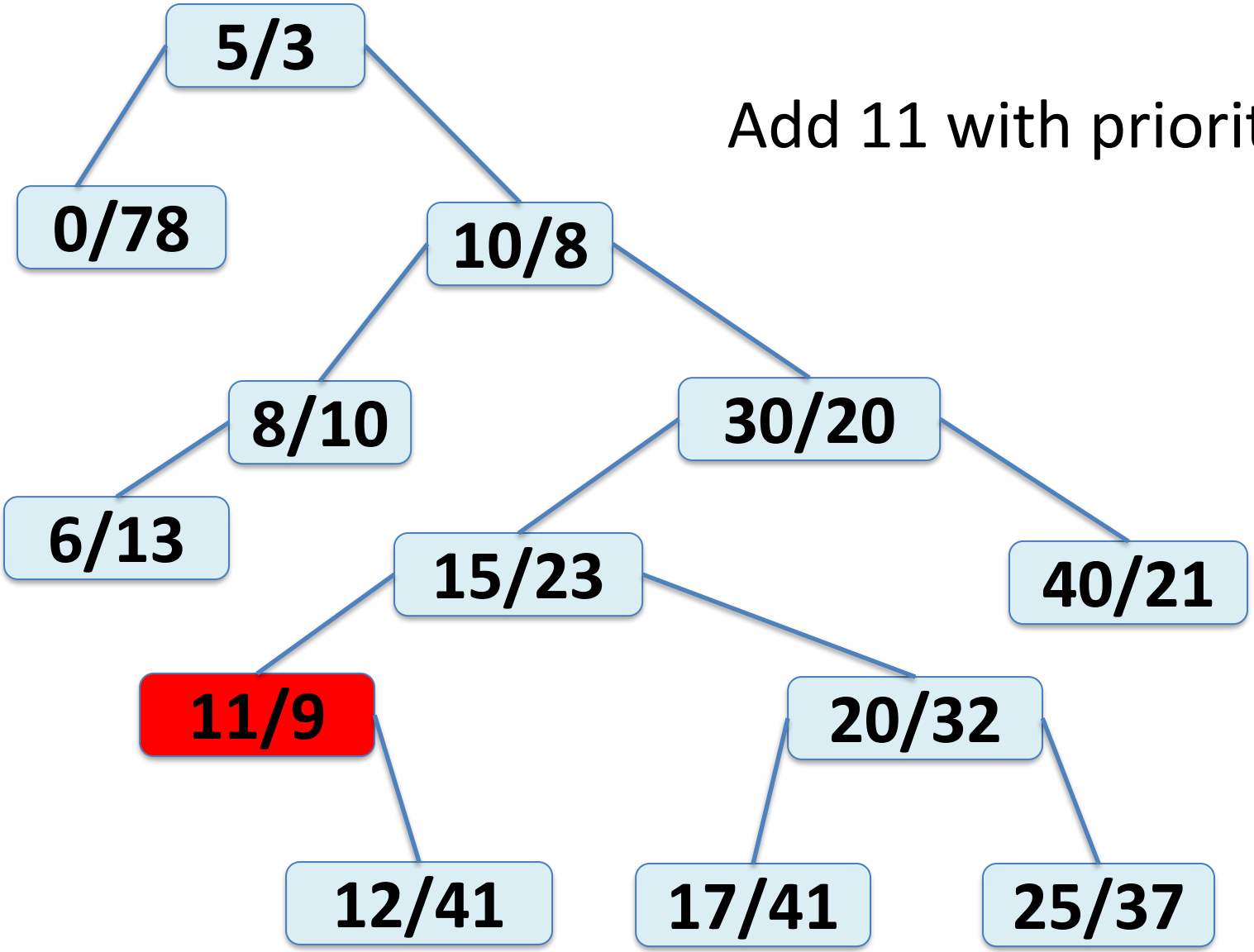
Add 11 with priority 9

TREAP



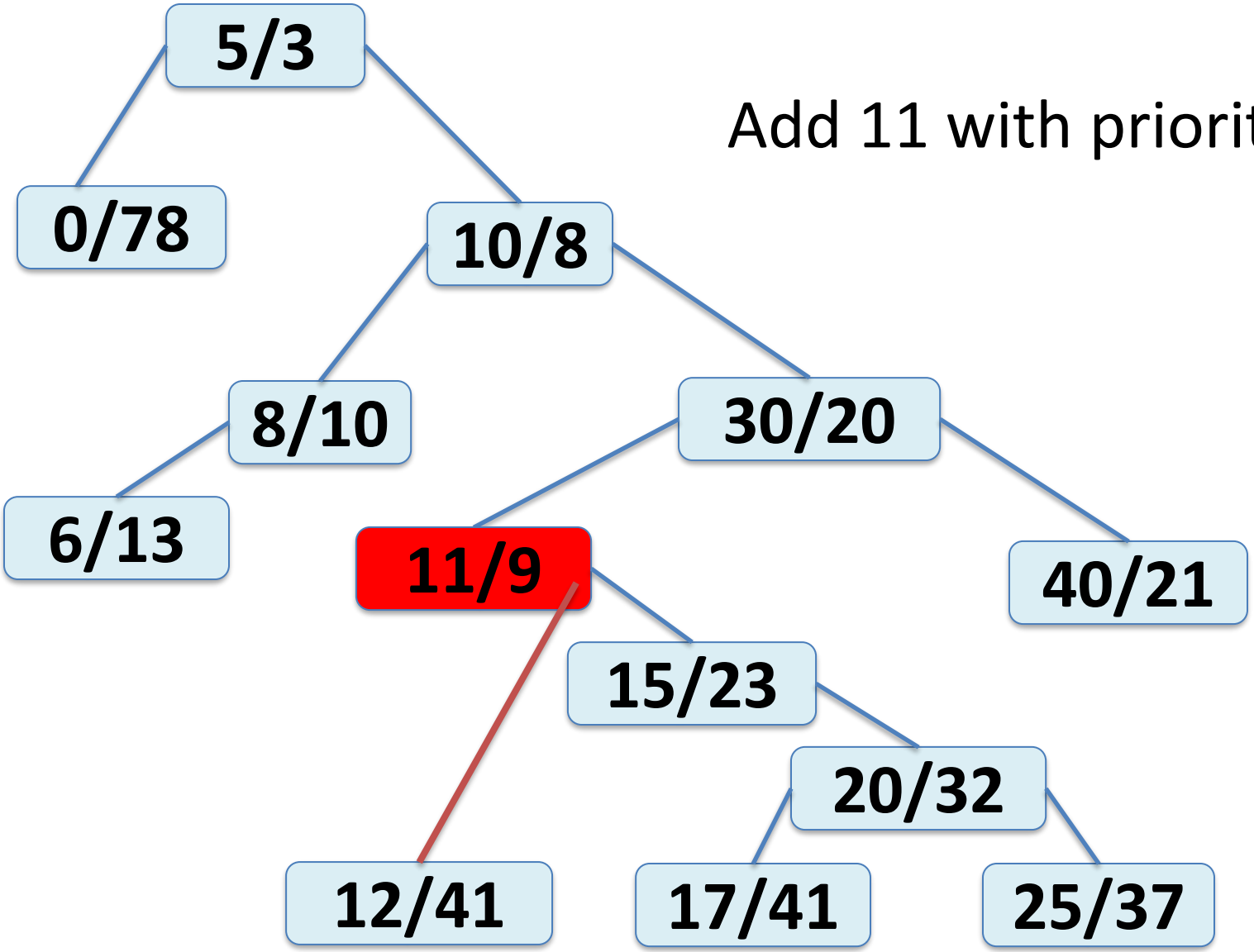
Add 11 with priority 9

TREAP



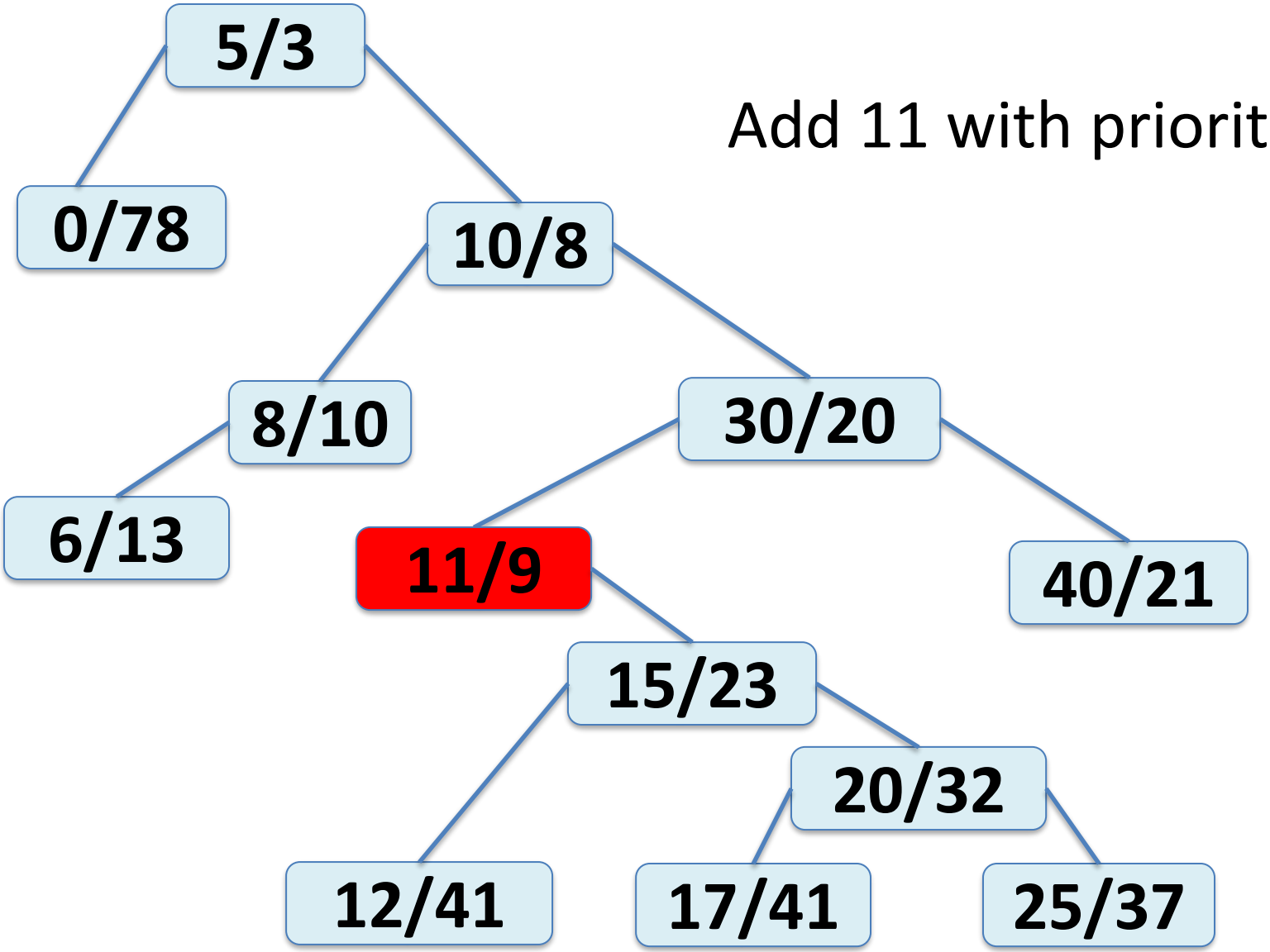
Add 11 with priority 9

TREAP



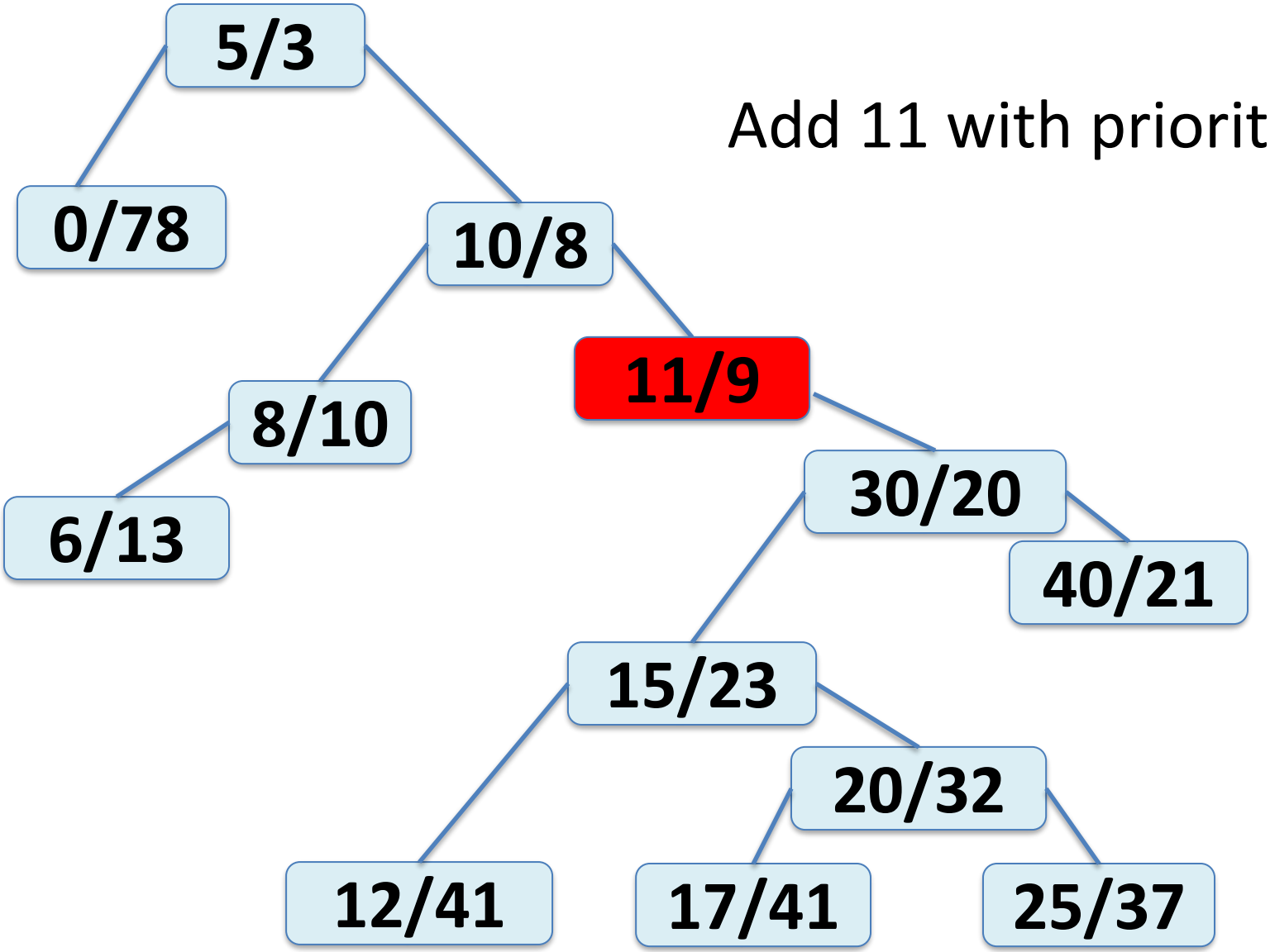
Add 11 with priority 9

TREAP



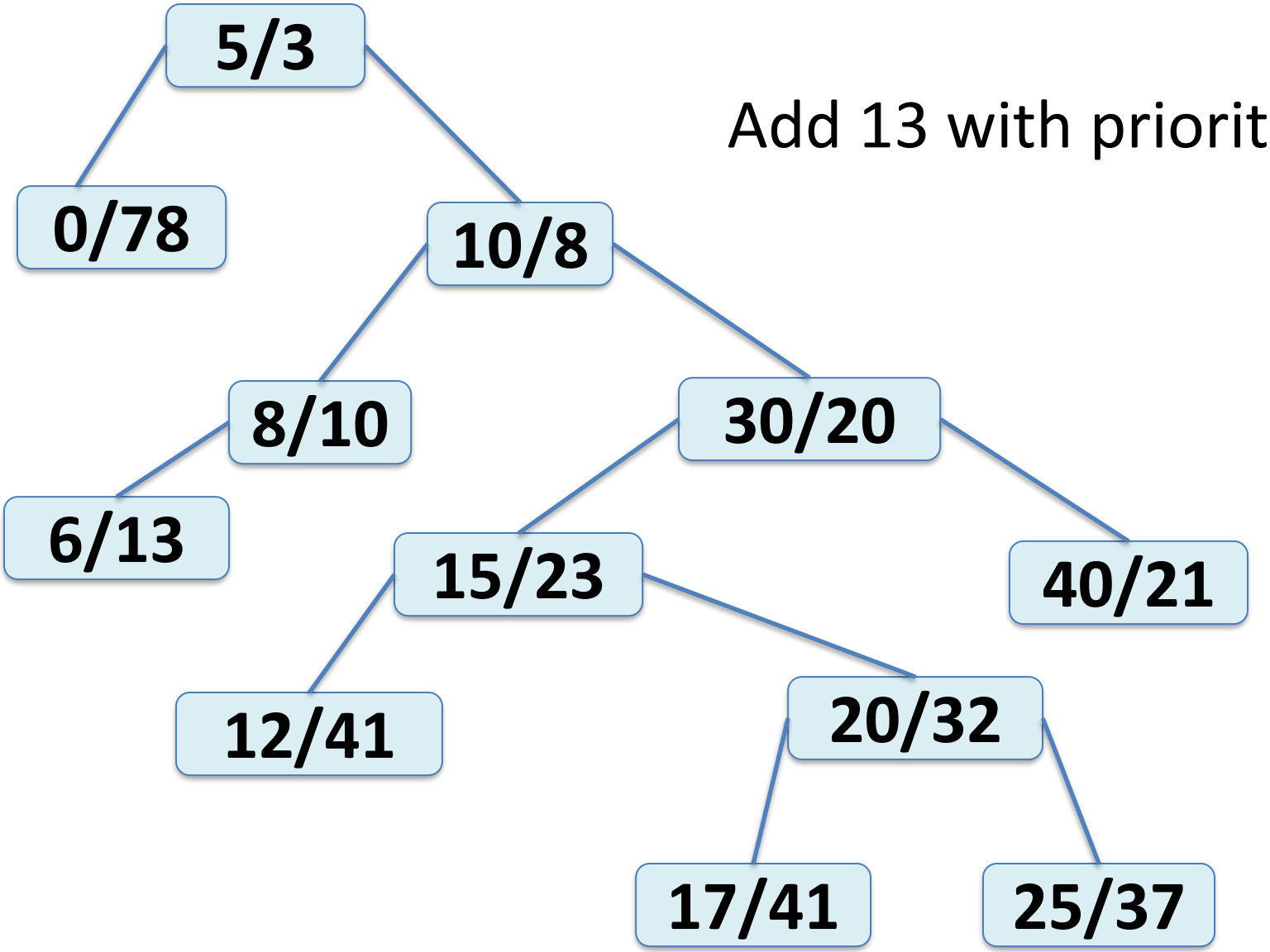
Add 11 with priority 9

TREAP



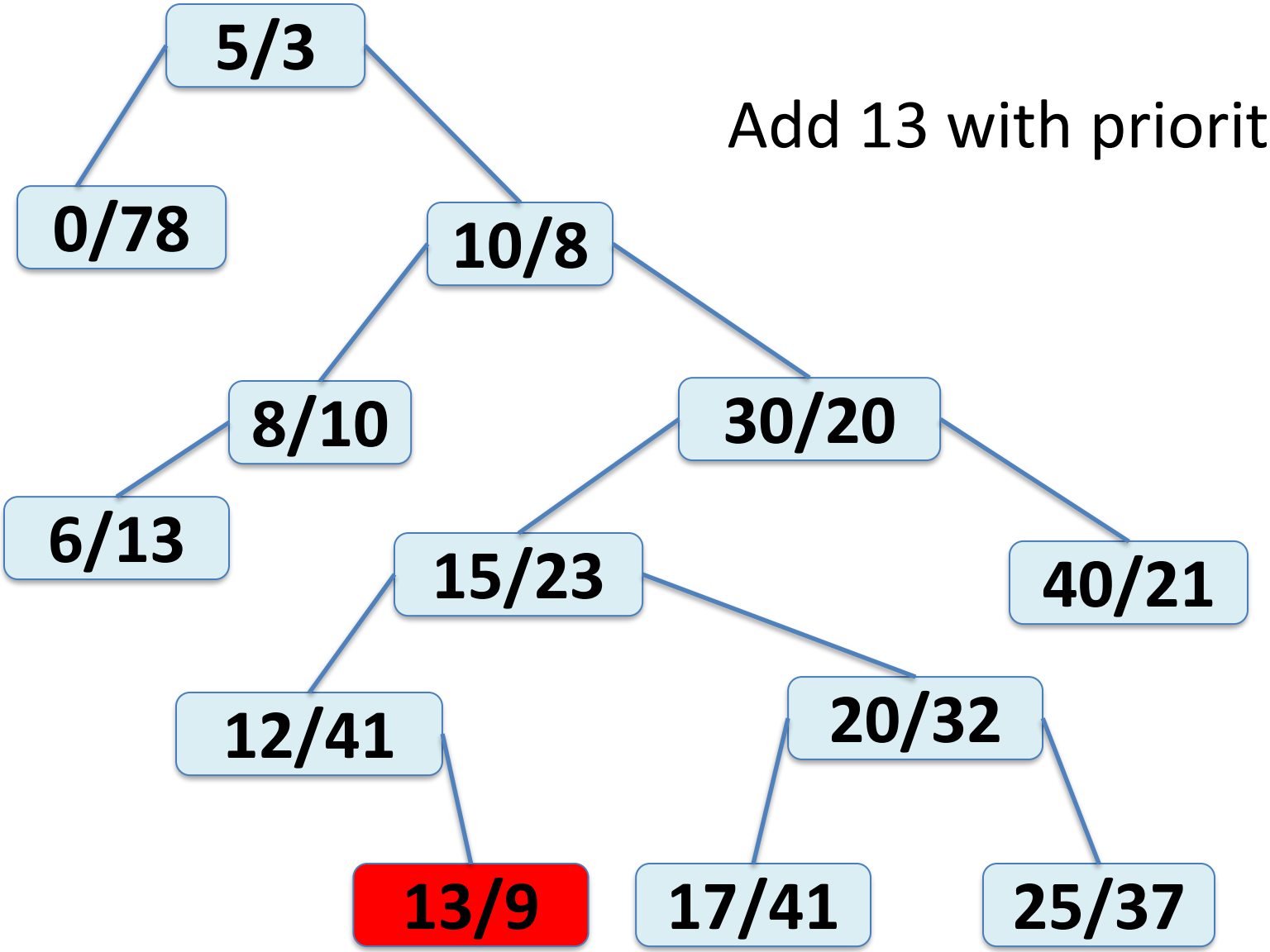
Add 11 with priority 9

TREAP



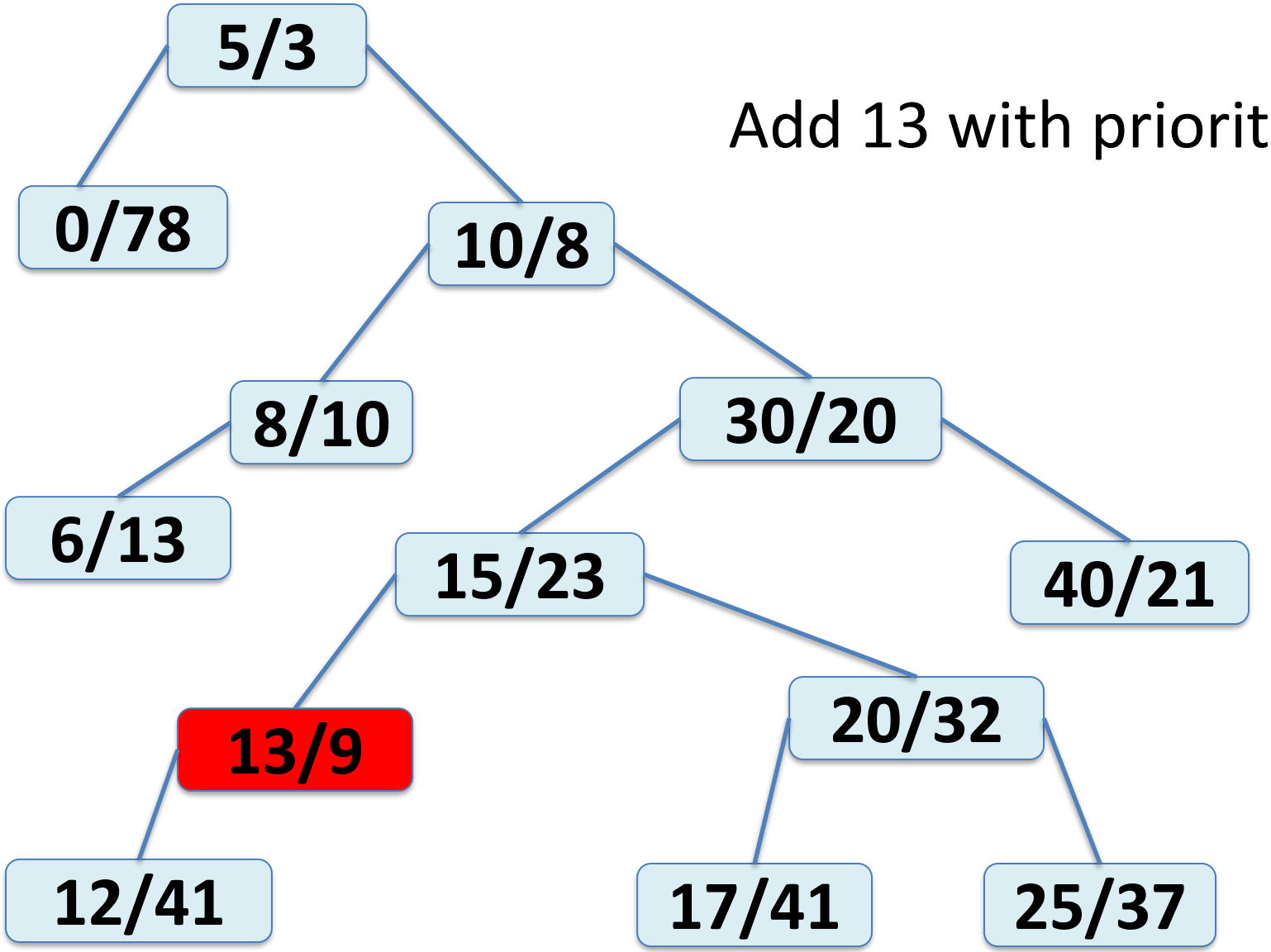
Add 13 with priority 9

TREAP



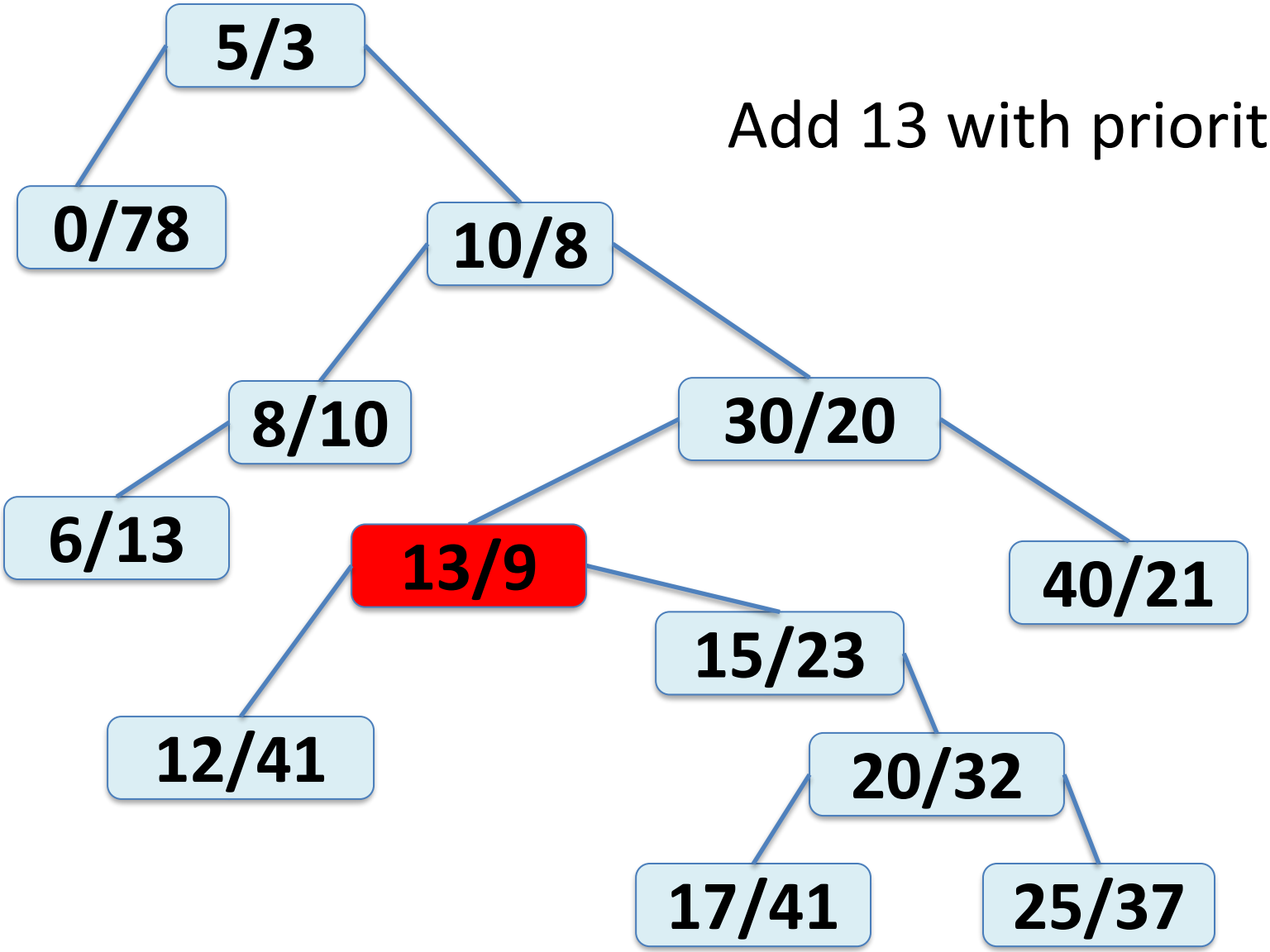
Add 13 with priority 9

TREAP



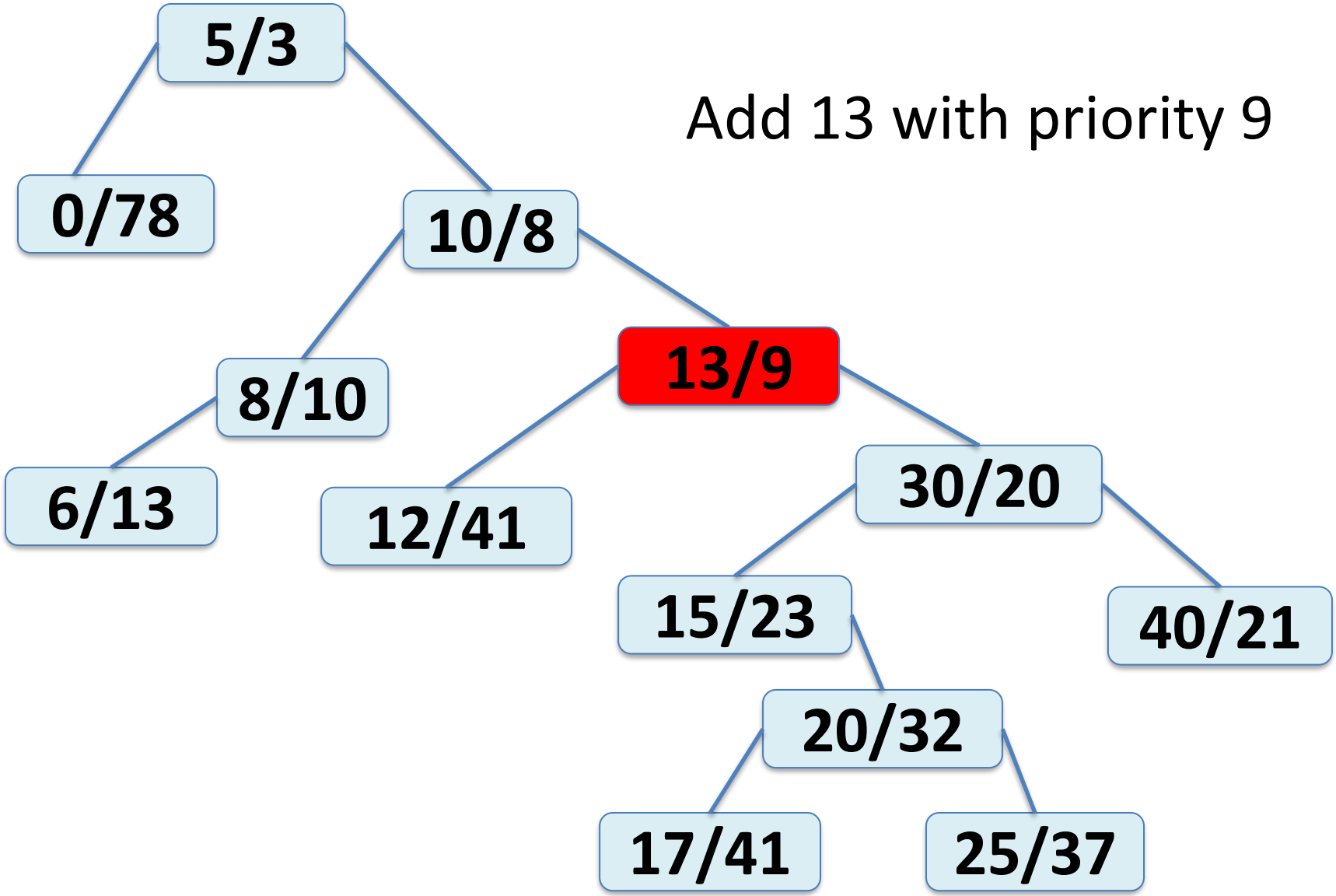
Add 13 with priority 9

TREAP



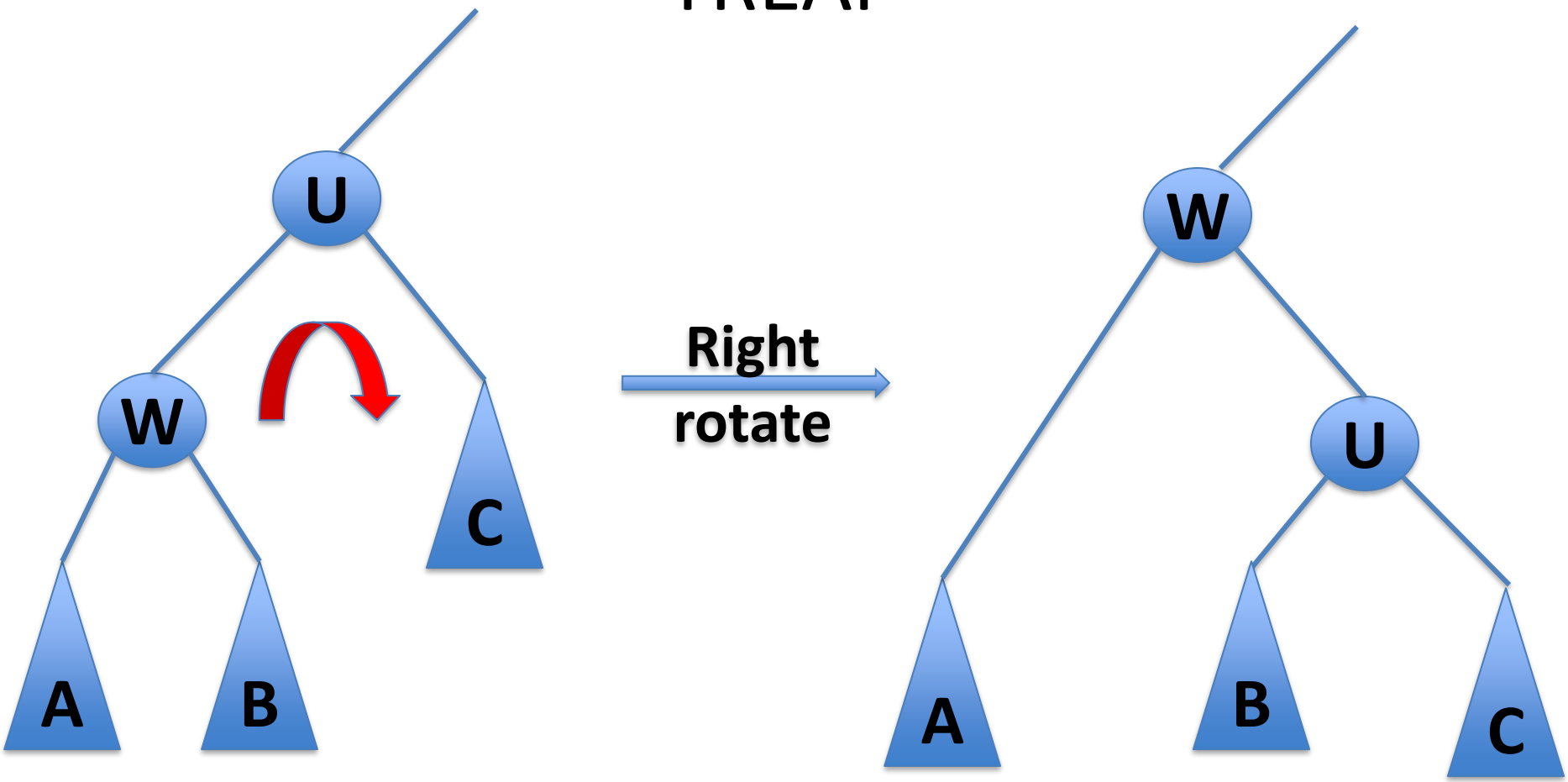
Add 13 with priority 9

TREAP

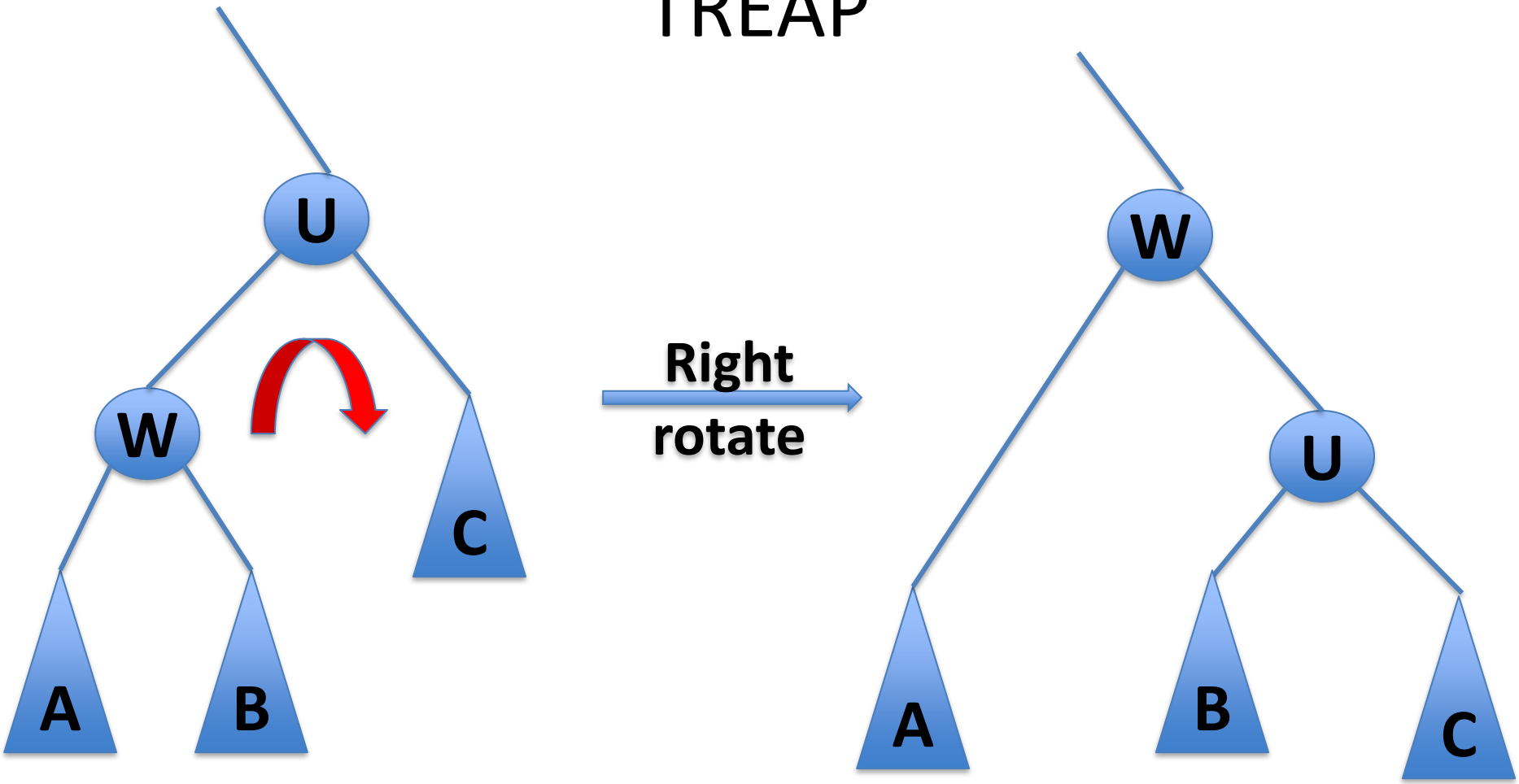


Add 13 with priority 9

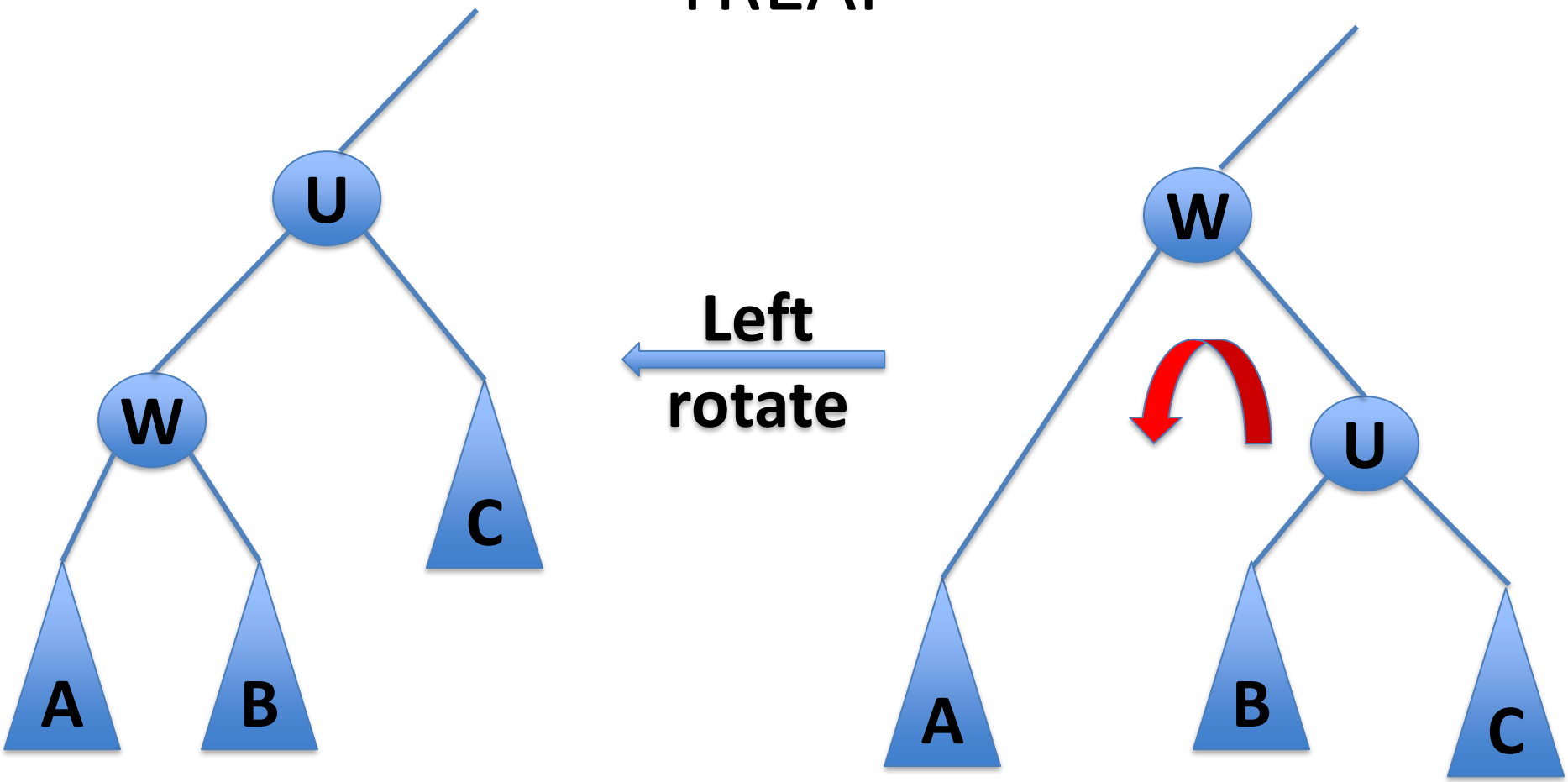
TREAP



TREAP



TREAP



TREAP

How do we add to a treap?

1. Insert as leaf (as usual for a binary search tree)

2. Rotate the new node up until the heap property of the treap is satisfied

What is the cost of adding to a treap?

TREAP

How do we remove from a treap?

1. Find the element to delete.

2. Rotate the node down in the tree until it is a leaf and then splice it out of the tree.

What is the cost of removing from a treap?

SKIPLIST vs TREAP

Expected length of search path in skiplist is at most

$$2\log(n) + O(1)$$

Expected length of search path in treap is at most

$$2\ln(n) + O(1) \approx 1.386\log(n) + O(1)$$